
Advanced System Applications and Practice for Internet and Multimedia

TM1117

Django

Instructor :

**Industrial Centre
The Hong Kong Polytechnic University**

What is Django?

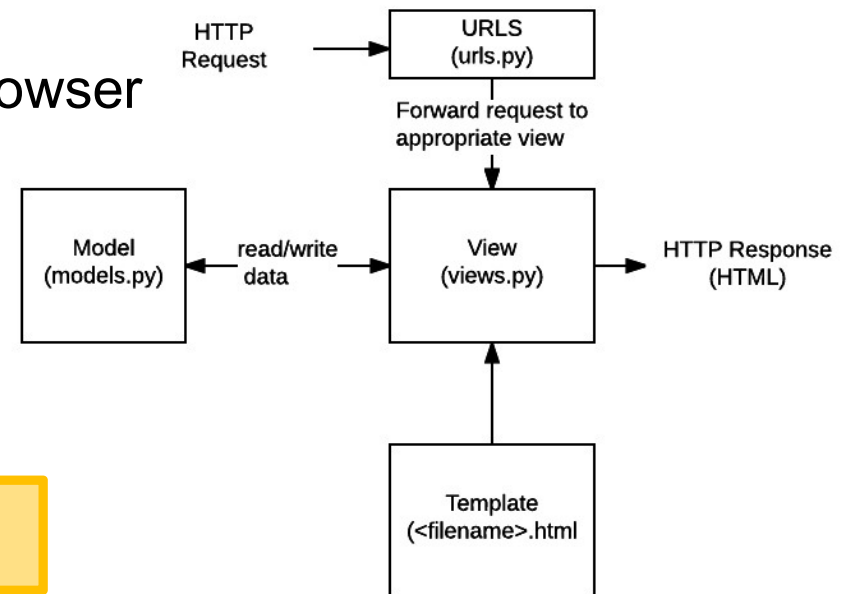
- High-level Python web framework that enables rapid development of secure and maintainable websites
- Developed between 2003 and 2005 by a web team who were responsible for creating and maintaining newspaper websites
- Free and open source

Why Django?

- **Complete**
 - Provides almost everything developers might want
- **Versatile**
 - Builds any type of website from content management systems to social networks to scientific computing platforms.
- **Secure**
 - Provides a secure way to manage user accounts and passwords, avoiding common mistakes
- **Scalable**
 - Uses a "shared-nothing" architecture, which means you can add hardware at any level – database servers, caching servers or Web/application servers
- **Maintainable**
 - Promotes the grouping of related functionality into reusable "applications"
- **Portable**
 - Is written in Python and runs on many platforms (Linux, Windows and Mac OS X)

Django web application

- 1) Wait for HTTP requests from the web browser
- 2) Base on the URL and possibly information in POST data or GET data
- 3) Read or write information from a database
- 4) Insert the retrieved data (database data) into placeholders in an HTML template (webpage)
- 5) Return a response to the web browser



Complete flow

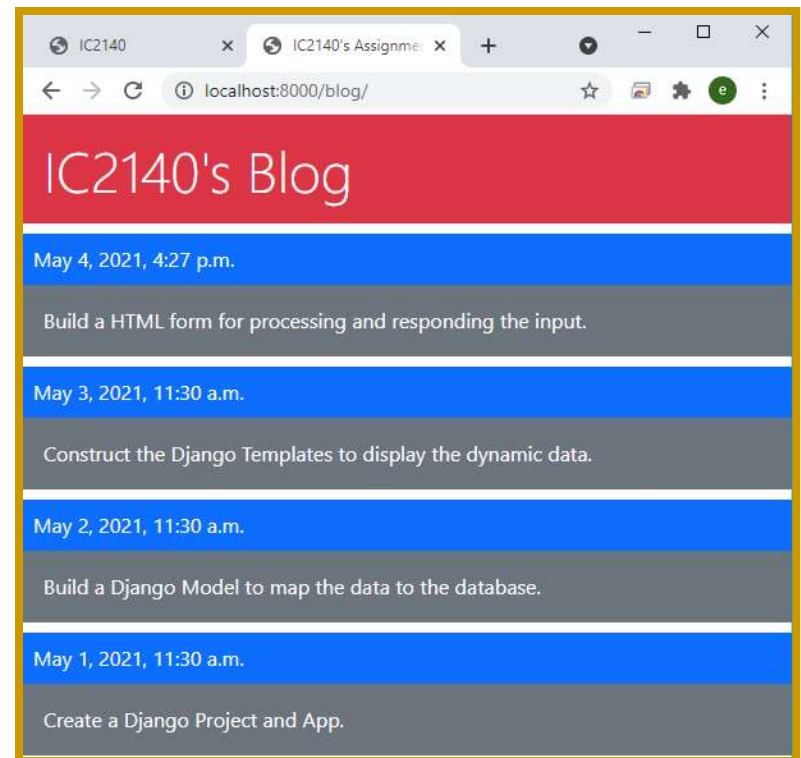
URL -> View -> Model (Typically) -> Template

Main features

- Forms
 - ❑ Collects user data for processing on the server
- User authentication and permissions
 - ❑ Includes a robust user authentication and permission system that has been built with security
- Caching
 - ❑ Provides flexible caching so that you can store all or part of a rendered page
- Administration site
 - ❑ Provides an admin page for site administrators to create, edit, and view any data models in your site.
- Serialising data
 - ❑ Serialises and serve your data as XML or JSON

Example: Create Blog website

- Allow people to view blog
- Add, change and delete blog in the administrative interface



Activate a virtual environment for django

- 1) Open the Anaconda Prompt
 - a) Start Menu -> Programs -> Anaconda3 (64-bit) -> Anaconda Prompt (Anaconda3)



```
Anaconda Prompt (Anaconda3)
(base) C:\>
```

- 2) Activated a virtual environment for django

```
$ cd /d c:\
$ django-env\Scripts\activate.bat
```

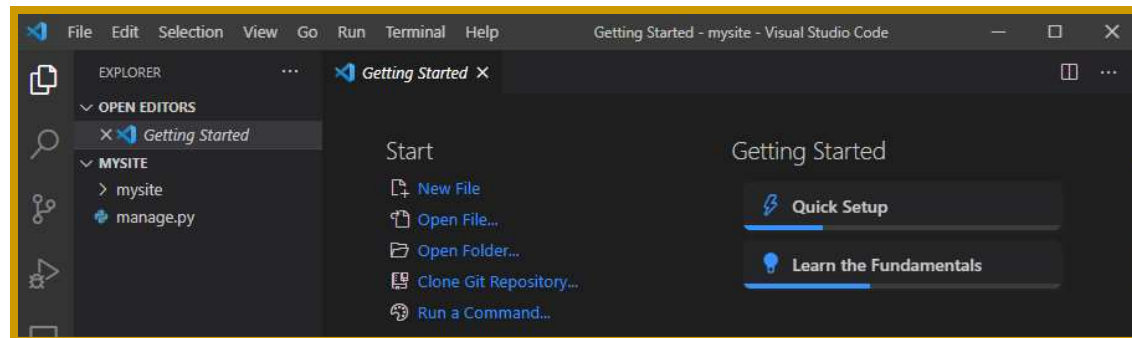


```
Anaconda Prompt (Anaconda3)
(django-env) (base) C:\>
```

Create Django Project

- 1) Change to directory (if not exist, create it) where you store the code
`$ cd /d D:\01-Home\workspace\IC2140\01-Blog`
- 2) Create a project "mysite"
`$ django-admin startproject mysite`
- 3) Navigate into the "mysite" directory
`$ cd mysite`
- 4) Open the project folder in the Visual Studio Code
 - a) `$ code .`
 - b) Click "Open Folder" -> Select the Project folder `D:\01-Home\workspace\IC2140\01-Blog\mysite\`

Install recommended extensions for python
"Python extension for Visual Studio Code"



Create Django Project

5) Set the timeZone to Hong Kong

- a) Open the mysite\mysite\settings.py file in the Visual Studio Code

6) Modify the code as below and save

```
TIME_ZONE = 'Asia/Hong_Kong'
```

7) Run the development web server in the Anaconda Prompt

\$ python manage.py runserver

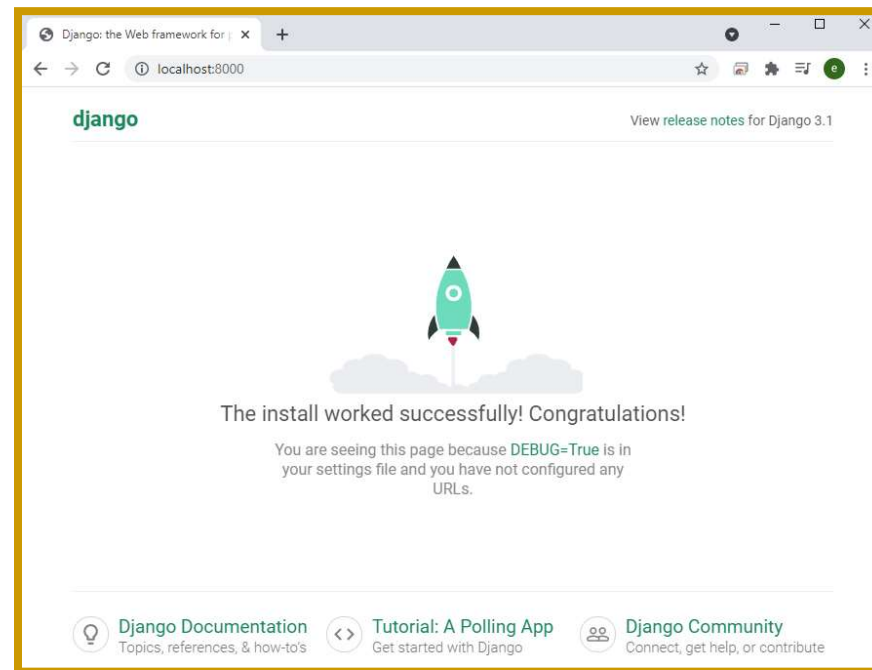
```
(django-env) (base) D:\01-Home\workspace\IC2140\01-Blog\mysite>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the
migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
April 27, 2021 - 17:48:17
Django version 3.1.7, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Create Django Project

- 8) Open the below link in the Chrome Browser
<http://localhost:8000/>
- ❑ The diango project "mysite" is working



Project file structure

- `manage.py`
 - Is a command-line utility and used to create applications, work with databases, and start the development web server.
- `mysite/__init__.py`
 - Empty file that tells Python to consider this directory as a Python package.
- `mysite/settings.py`
 - Contains all the website settings (E.g. the location of our static files, database configuration details and etc.)
- `mysite/urls.py`
 - Defines the site URL-to-view mappings and contains the list of all the endpoints.
- `mysite/asgi.py`
 - Standard for Python asynchronous web apps and servers to communicate with each other
 - A succeeded interface to the WSGI
- `mysite/wsgi.py`
 - Help the Django application communicate with the webserver and Use for deploying the application on to servers like Apache etc.

```
mysite/  
    manage.py  
    mysite/  
        __init__.py  
        settings.py  
        urls.py  
        asgi.py  
        wsgi.py
```

Enable Django Admin

- 1) Shutdown the django server in the Anaconda Prompt
 - a) Press CTRL + C
- 2) Create the database for storing the user account. E.g. superuser
\$ python manage.py migrate

```
(django-env) (base) D:\01-Home\workspace\IC2140\01-Blog\mysite>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
```

Enable Django Admin

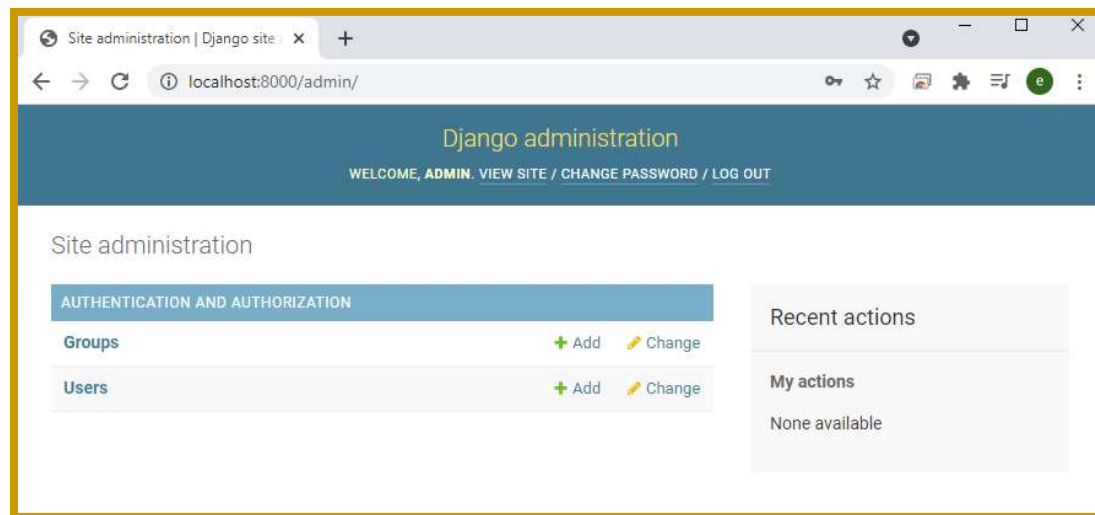
- 3) Create a superuser "admin", who can login in the admin site, in the Anaconda Prompt

```
$ python manage.py createsuperuser
>> Username: admin
>> Email address: admin@admin.com
>> Password: admin
>> Password (again): admin
>> Bypass password..... ? y
```

```
(django-env) (base) D:\01-Home\workspace\IC2140\01-Blog\mysite>python manage.py createsuperuser
Username (leave blank to use 'iclwong'): admin
Email address: admin@admin.com
Password:
Password (again):
The password is too similar to the username.
This password is too short. It must contain at least 8 characters.
This password is too common.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Enable Django Admin

- 4) Run the development web server in the Anaconda Prompt
\$ python manage.py runserver
- 5) Open the below link in the Chrome Browser
<http://localhost:8000/admin>
- 4) Login the "Django administration" with the superuser
Username: admin
Password: admin



Django Application (app)

- **Django Project** refers to the entire application and all its parts
- **Django Application**(app) refers to a submodule of the project and is a component that together make up your project. It provides some set of features. It is portable and applies in multiple projects.
- For example,
 - ❑ Project: Online Shopping website
 - ❑ Application (app): Cart, Products and Profile

```
my_shopping_site/  # project name
manage.py
products/          # app 1
cart/              # app 2
profile/           # app 3
my_shopping_site/
```

Project: Facebook website
App: ???

Django Application (app)

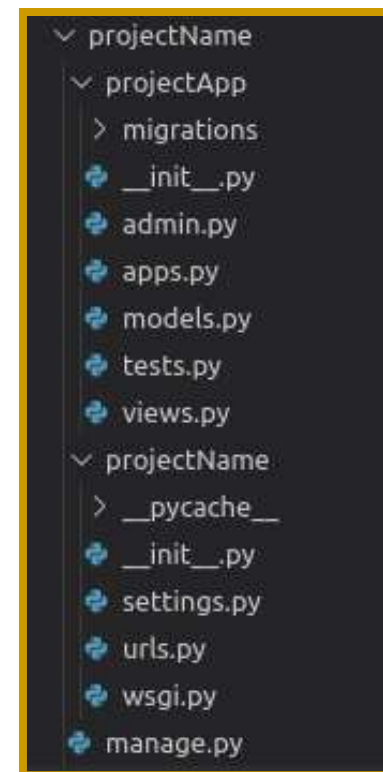
- Organize the website into apps
- Apps can be used in multiple different websites/projects and a website can have multiple apps
- Has its own data (models) and webpages (views)
- Example:
 - A product in the online shopping websites
 - Views (webpages)
 - Display the product in different catalogues
 - Models (data)
 - Product information such as name, type and price

Django Application (app)

- Make a folder for the app in the project directory
- Mainly contains
 - ❑ model.py
 - ❑ views.py
 - ❑ urls.py

Complete flow

URL -> View -> Model (Typically) -> Template



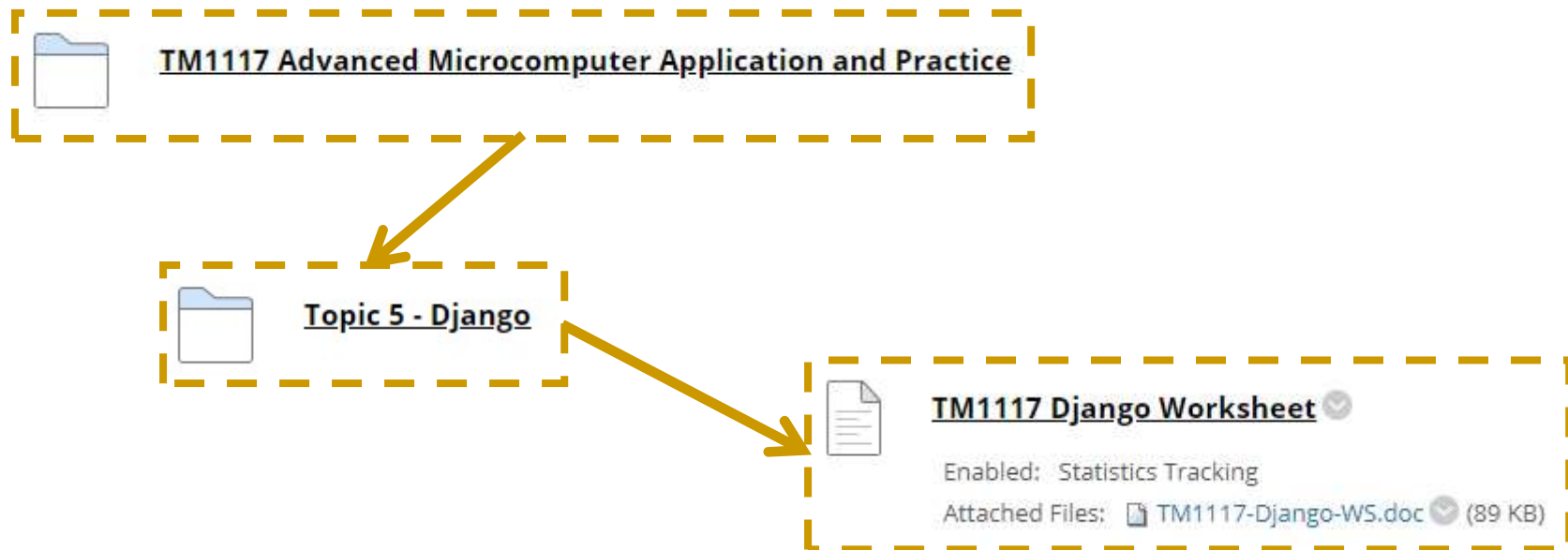
Create "blog" application(app)

- 1) Shutdown the django server in the Anaconda Prompt
 - a) Press CTRL + C
- 2) Create "blog" application(app)
\$ python manage.py startapp blog
- 3) Register the "blog" app to the project in Visual Studio Code
 - a) Open the mysite\mysite\settings.py file in the Visual Studio Code
 - b) Search the "INSTALLED_APPS" location
 - c) Add the "blog" app to "INSTALLED_APPS" variable

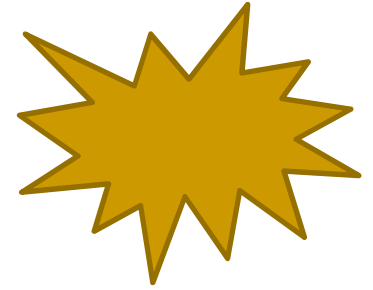
```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'blog',  
]
```

Django Worksheet

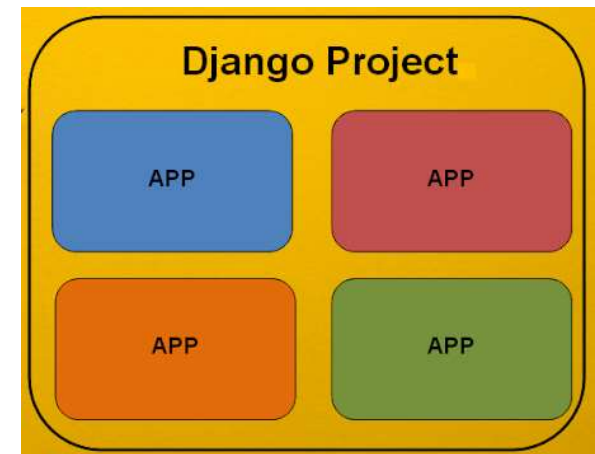
- Download it from the Blackboard



Worksheet - Task 1:



- Develop the online inventory management system
 - Suggest the features that are essential for the system
 - E.g. Inventory list
 - Suggest the Django app with associated function for the features you suggested
 - E.g. inventory (app) – Allow users to view the inventory and update the inventory information



Django Model

Django Model

- Implement as Python classes in the `models.py` file
- Django uses it to access and manage data for making dynamic website
- Define the structure of stored data, including the field types and the maximum size
- Allow to access/change a database by calling functions on the models
- Implement as subclasses of `django.db.models.Model`, and can include fields, methods and metadata.

```
class MyModelName(models.Model): #Define the model

    # Fields
    my_field_name = models.CharField(max_length=20)

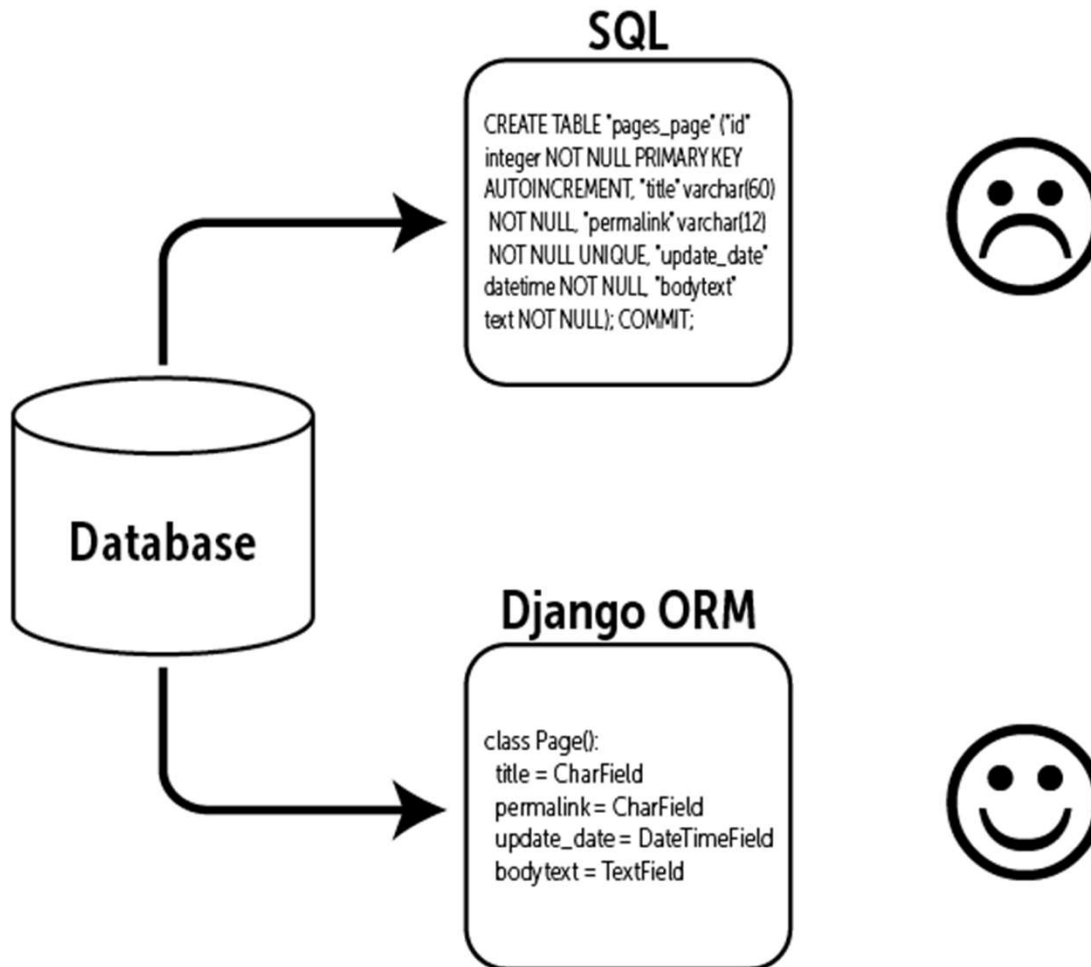
    # Metadata
    class Meta:
        ordering = ['-my_field_name']

    # Methods
    def get_absolute_url(self): #Return the url
        return reverse('model-detail-view', args=[str(self.id)])

    def __str__(self): #Represent the model name in Admin site.....
        return self.my_field_name
```

Reference: <https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django/Models>

Django Model



Create Model

- 1) Create the model object "Entry" in the [mysite/blog/models.py](#)
 - a) Open the [models.py](#) in Visual Studio Code
 - b) Create a table "Entry" and 2 attributes "**text**" and "**date_posted**"
 - c) Create a **__str__()** method to represent the object or model

```
class Entry(models.Model):  
    #Fields  
    text = models.TextField()  
    date_posted = models.DateTimeField(auto_now_add=True)  
  
    #Methods  
    def __str__(self):  
        return 'Entry {}'.format(self.id)
```

TextField()
- A large text field that can hold many thousands of characters
DateTimeField()
- Python datetime object

The **__str__()** method is the default human-readable representation of the object. Django uses it in the administration site and some places.

Register Model

- 1) Register the model object "Entry" in the [mysite/blog/admin.py](#)
 - a) Open the [admin.py](#) in Visual Studio Code
 - b) Import the Entry package (**Brown arrow**) at the beginning of the admin.py



```
from django.contrib import admin  
from .models import Entry
```

- c) Register the Entry model in the [admin.py](#) after the import statements

```
# Register your models here.  
admin.site.register(Entry)
```

Add Model to the database

- 1) Synchronize the model into the database by running migrations to reflect the changes in the database
 - a) Go to the Anaconda Prompt
\$ python manage.py makemigrations

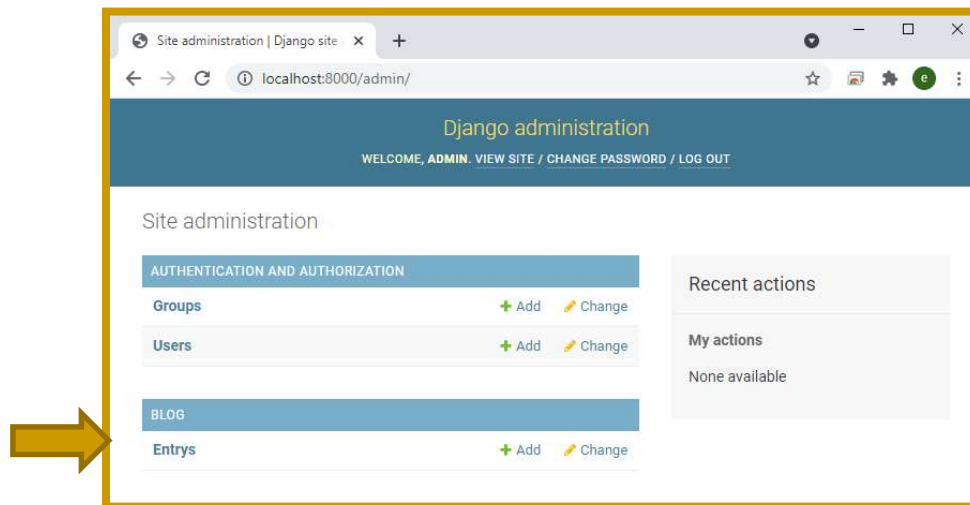
```
(django-env) (base) D:\01-Home\workspace\IC2140\01-Blog\mysite>python manage.py makemigrations
Migrations for 'blog':
  blog\migrations\0001_initial.py
  - Create model Entry
```

\$ python manage.py migrate

```
(django-env) (base) D:\01-Home\workspace\IC2140\01-Blog\mysite>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
  Applying blog.0001_initial... OK
```

Add Model to the database

- 2) Run the development web server in the Anaconda Prompt
\$ python manage.py runserver
- 3) Open the below link in the Chrome Browser
<http://localhost:8000/admin>
- 4) Login the "Django administration" with the superuser
Username: admin
Password: admin



Declare metadata for Entry model

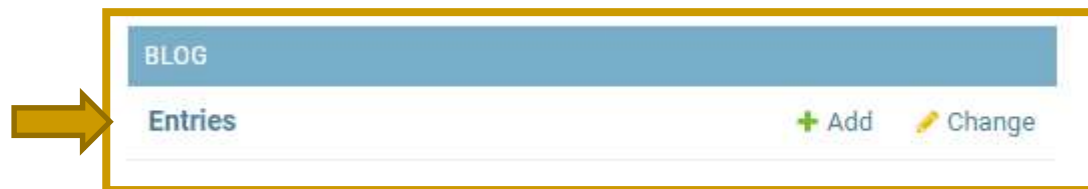
- 1) Set the plural form of Entry word in the [mysite/blog/models.py](#)

- a) Open the models.py in Visual Studio Code
- b) Add the following code to the [models.py](#) within the "Entry" Class

```
class Meta:  
    verbose_name_plural = 'entries'
```

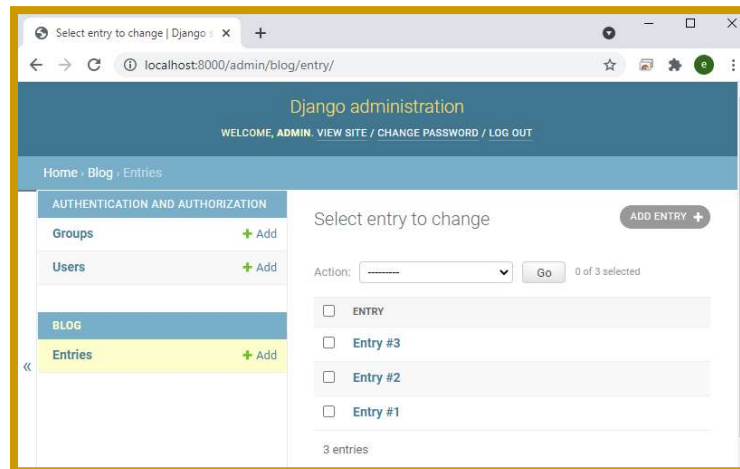
metadata
- Control the default
ordering of records
returned

- 2) Refresh the web page. It shows "Entries", not "Entrys"



Add 3 "Entry" data in the Django Admin site

- 1) Create 1st entry
 - a) Click "Add" button in the Django Admin site
 - b) Input the text "Create a Django Project and App." in the Text area
 - c) Click "Save" button
- 2) Repeat the step 1 to create 2nd and 3rd entries
 - 1) 2nd entry: Build a Django Model to map the data to the database.
 - 2) 3rd entry: Construct a Django Template to display the dynamic data.



Django Database

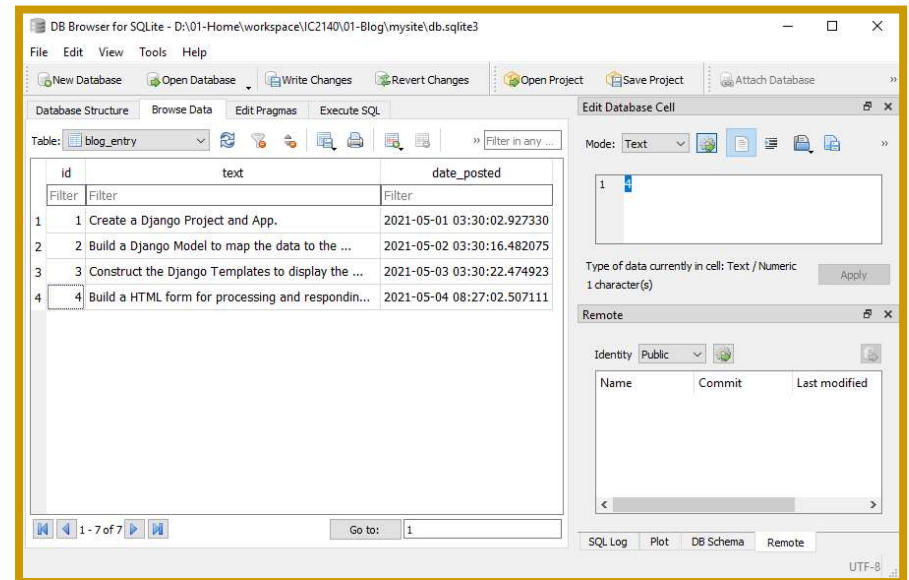
- Default database: SQLite

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

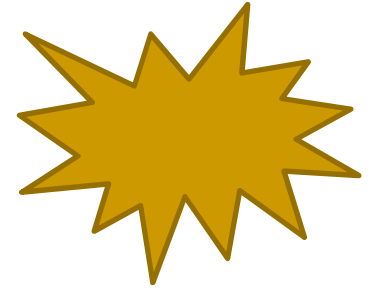
- The file "db.sqlite3" is the database
- Supports five databases:
 - ❑ PostgreSQL
 - ❑ MySQL
 - ❑ SQLite
 - ❑ Oracle
 - ❑ MariaDB

SQLite Database Browser Portable

- Check out the Django model in the database browser
 - a) Open the software "SQLite Database Browser"
 - b) Click "Open Database" and select "db.sqlite3" under the [D:\01-Home\workspace\IC2140\01-Blog\mysite](#) folder
 - c) Click the "Browse Data" Tab
 - d) Select "blog_entry" as the Table



Worksheet - Task 2:



- 1) Design the Django Model for the inventory data
 - ❑ Inventory no. : E1139793
 - ❑ Location: W311F
 - ❑ Owner: ICDAVID
 - ❑ Description: Philips 220B2CB 22" Monitor
 - ❑ Price: \$1380
 - ❑ Purchase Date: 30 AUG 2010
 - ❑ Active: True
- Model field reference
 - ❑ <https://docs.djangoproject.com/en/3.2/ref/models/fields>

Worksheet - Task 2:

2) Design the Django Model for the inventory owner

- ❑ Owner: ICDAVID
- ❑ Full Name: David Chan
- ❑ Title: Engineer
- ❑ Phone: 27665354
- ❑ Email: icdavid@polyu.edu.hk

■ Model field reference

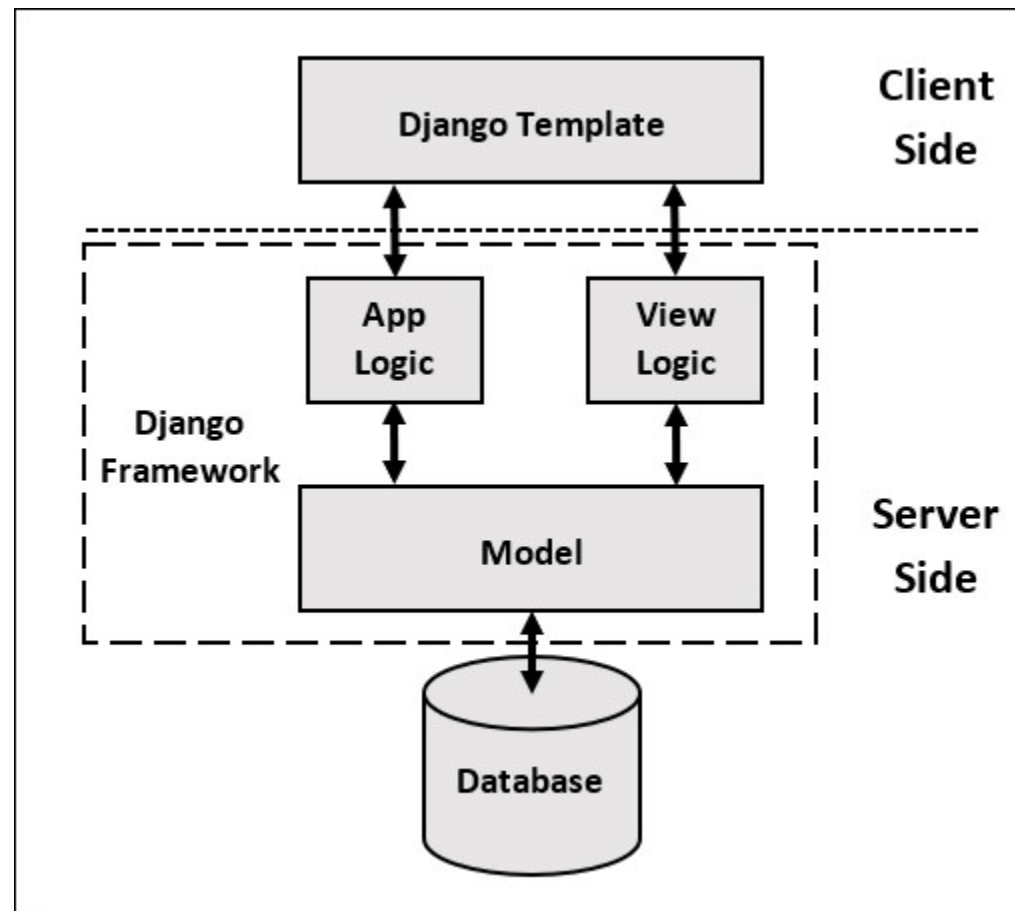
- ❑ <https://docs.djangoproject.com/en/3.2/ref/models/fields>

Django View

Django View

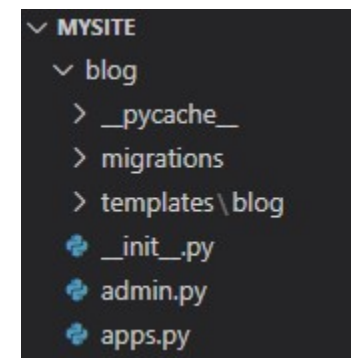
- Implement as Python function to make the web page user can see
 - Make use of models for retrieving database data
 - Use HttpRequest object as input
 - Contains useful information about the client, sessions, and etc.
 - Use HttpResponse object as output
 - Fill in templates and render the HTML output
-

Django View



Create index page (home page)

- Django will look for templates in a subdirectory named "templates" of your applications.
 - Template defines the structure or layout of a file (such as an HTML page), it uses placeholders to represent actual content.
-
- 1) Create a folder "templates" under the [mysite/blog](#) folder in the Visual Studio Code
 - a) Right-click the "blog" folder in Visual Studio Code
 - b) Select the "New folder" and assign a name "templates" to it
 - 2) Create a folder "blog" under the [mysite/blog/templates](#) folder in the Visual Studio Code
 - a) Right-click the "templates" folder in Visual Studio Code
 - b) Select the "New folder" and assign a name "blog" to it



Create index page (home page)

- 3) Create a file "index.html" under the [mysite/blog/templates/blog](#) folder
 - a) Right-click the "blog" folder in Visual Studio Code
 - b) Select the "New file" and assign a name "index.html" to it
- 4) Put the below HTML code (static page) to [mysite/blog/templates/blog/index.html](#) file

```
<HTML>
  <head>
    <title>IC2140</title>
  </head>
  <body>
    <h1>IC2140</h1>
  </body>
</HTML>
```

Hyper Text Markup Language (HTML)

- Used to create web pages and web applications

```
<HTML>
  <head>
    <title>IC2140</title>
  </head>
  <body>
    <h1>IC2140</h1>
  </body>
</HTML>
```

<html>: HTML document that informs the browser

<head>: Information about the document (metadata)

<title>: The title of HTML page that appears at the top of the browser window

<body>: The body content of the page that is visible to the end user

<h1>: The 1st level of heading of the webpage

Render index page (home page)

- 1) Render the "index.html" page when requested by user
 - a) Open the `mysite/blog/views.py` file in the Visual Studio Code
 - b) Add the following code in it

```
def index(request):  
    return render(request, 'blog/index.html')
```

views.py

1. Processes an HTTP request
2. Fetches the required data from the database
3. Renders the data in an HTML page using an HTML template
4. Returns the generated HTML in an HTTP response to display the page to the user

Map URL to index page (home page)

- 1) Create a file "urls.py" under the [mysite/blog/](#) folder
 - a) Right-click the blog folder in Visual Studio Code
 - b) Select the "New file" and assign a name "urls.py" to it
 - c) Paste the following code in it

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index),
]
```

```
path("", views.index),
```

- "" is a URL pattern. It is an empty string.

- `views.index` function will be called if the URL pattern is detected. It is the function name `index()` in the `views.py` file

Map URL to index page (home page)

2) Update the routing in `mysite/mysite/urls.py` file (Entry point of the Project)

a) Open the `mysite/mysite/urls.py` in the Visual Studio Code

b) Search below "import" statement and update it

```
from django.urls import path, include
```

c) Append the "urlpatterns" variable with the following code (**Brown arrow**)



```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('blog.urls')),      #Include app "blog/urls.py"  
]
```

When a URL that starts with " " empty string is received, the URLConf module `blog.urls` will process the remaining substring.

Access the index page in browser

- 1) Start the server in the Anaconda Prompt (if stop)
`$ python manage.py runserver`
- 2) Open the link as below in the Chrome Browser
`http://localhost:8000/`



Django Template

Django Template

- Django template is a text file
- Template tag is surrounded by `{%` and `%}`
- Template variable is surrounded by `{{` and `}}` and passed to the template at runtime

- Display Logic. E.g., `{% if %}...{% endif %}`
- Loop Control. E.g., `{% for x in y %}...{% endfor %}`
- Block Declaration. E.g., `{% block content %}...{% endblock %}`
- Content Import. E.g., `{% include "header.html" %}`
- Inheritance. E.g., `{% extends "base.html" %}`

- Simple Variables. E.g., `{{ title }}`
- Object Attributes. E.g., `{{ page.title }}`
- Dictionary Lookups. E.g., `{{ dict.key }}`
- List Indexes. E.g., `{{ list_items.0 }}`
- Method Calls. E.g., `{{ var.upper }}`, `{{ mydict.pop }}`

Reference: <https://djangobook.com/mdj2-django-templates/>

Django Template

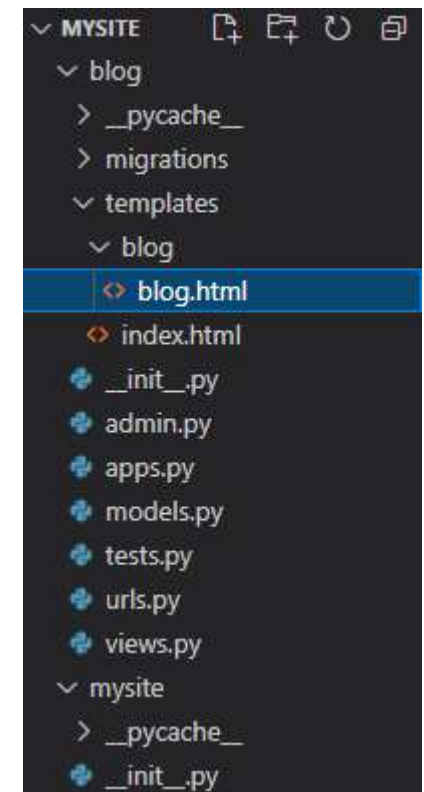
- Filters modify a variable for display. You apply a filter to a variable using the **|** (pipe) character. Here are some examples.

- Change Case. E.g., `{{ name|title }}` or `{{ units|lower }}`
- Truncation. E.g., `{{ post_content|truncatewords:50 }}`
- Date Formatting. E.g., `{{ order_date|date:"D M Y" }}`
- List Slicing. E.g., `{{ list_items|slice:"3" }}`
- Default Values. E.g., `{{ item_total|default:"nil" }}`

- Define the decimal places

Create blog page

- 1) Create a folder "blog" under the `mysite/blog/templates` folder
 - a) Right-click the "templates" folder in Visual Studio Code
 - b) Select the "New folder" and assign a name "blog" to it
- 2) Create a file "blog.html" under the `mysite/blog/templates/blog` folder
 - 1) Right-click the blog folder in Visual Studio Code
 - 2) Select the "New file" and assign a name "blog.html" to it



Create blog page

- 3) Put the below HTML code to <mysite/blog/templates/blog/blog.html> file

```
<HTML>
  <head>
    <title>IC2140's Assignment 1</title>
  </head>
  <body>
    <h1>IC2140's Blog</h1>
  </body>
</HTML>
```

Render blog page

- 1) Render the "blog.html" page when requested by user
 - a) Open the `mysite/blog/views.py` file in the Visual Studio Code
 - b) Add the following code in it

```
def blog(request):  
    return render(request, 'blog/blog.html')
```

Map URL to blog page

- 1) Open the file "urls.py" under the [mysite/blog/](#) folder in Visual Studio Code
 - a) Append the "urlpatterns" variable with the following code



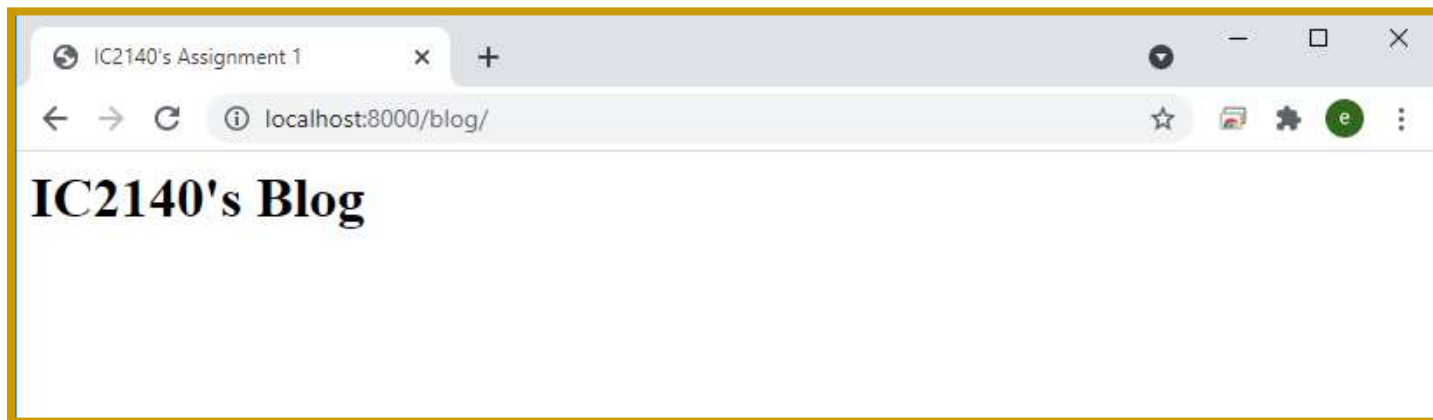
```
urlpatterns = [  
    path('', views.index),  
    path('blog/', views.blog),  
]
```

path('blog/', views.blog),

- 'blog/' is a URL pattern. It is a URL starts with blog/.
- views.blog function will be called if the URL pattern is detected. It is the function name blog() in the views.py file

Access the blog page in browser

- 1) Start the server in the Anaconda Prompt (if stop)
\$ python manage.py runserver
- 2) Open the link as below in the Chrome Browser
<http://localhost:8000/blog>



Extract entries data from database

- 1) Import the model class that we can access data "entries" in the views
 - a) Open the [mysite/blog/views.py](#) file in the Visual Studio Code
 - b) Add the 2nd statement (below code) in it



```
from django.shortcuts import render
from .models import Entry
```

Extract entries data from database

2) Retrieve all entries from the database

- a) Open the [mysite/blog/views.py](#) file in the Visual Studio Code
- b) Update the `blog()` method with the following code

```
def blog(request):  
    entries = Entry.objects.all()  
    context = {'entries' : entries}    # Store the data in "context" dictionaries  
    return render(request, 'blog/blog.html', context) # Pass the context to HTML template
```

Entry.objects.all()

- Get a list of Entry objects

context = {'entries' : entries}

- The context variable is a python dictionary. It contains the data to insert into the placeholders

render()

- Create an HTML page "blog.html" and return the page as a response

Display entries in the blog page

1) Insert the entries in the blog page

- 1) Open the [mysite/blog/templates/blog/blog.html](#) file
- 2) Place the following code after `<h1>.....</h1>` block

```
{% for entry in entries %}
<p>
<strong>{{ entry.date_posted }} </strong>
<br>{{ entry.text }}
</p>
{% endfor %}
```

{% %}

- Template tag (function) is used to loop through lists, perform conditional operations based on the value of a variable.

{{ }}

- Template variable is a placeholders for the information from the view that we want to include.

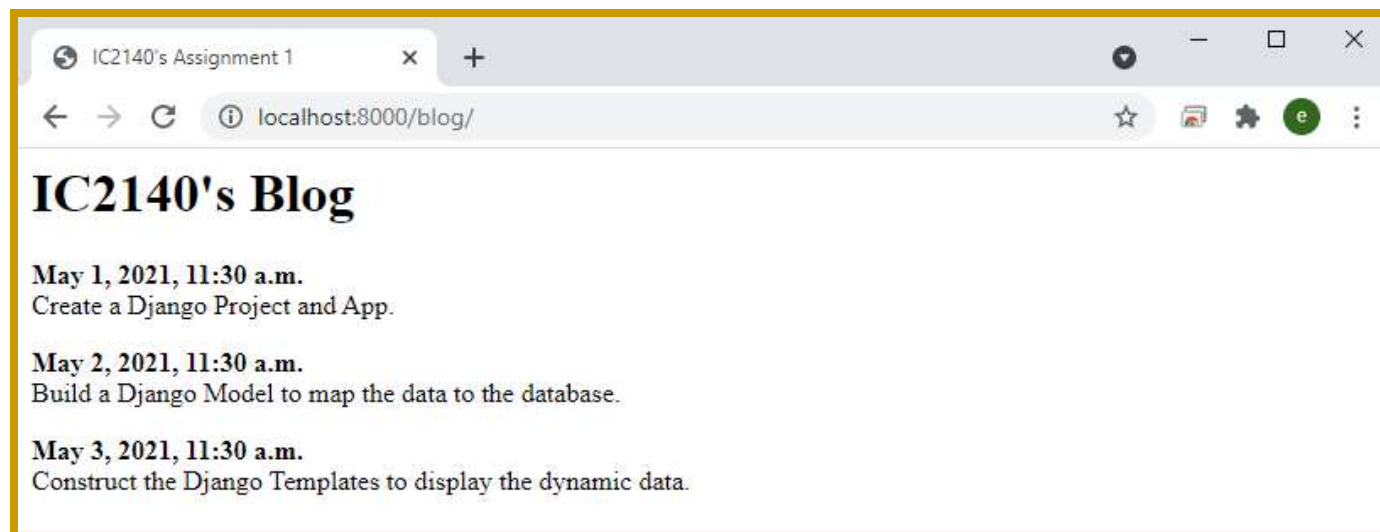
E.g. context = {'entries' : entries}

- The variables are named with the **keys**. When the template is rendered by the render() function in the views.py, the variables are replaced with their **associated values**.

Display entries in the blog page

- 2) Open the below link in the Chrome Browser
(Assume that the server is running)

<http://localhost:8000/blog>



Display the entries in descending order

- 1) Retrieve all entries from the database in descending order
 - a) Open the [mysite/blog/views.py](#) file in the Visual Studio Code
 - b) Search the "Entry.objects.all()" location
 - c) Update it with the following code

```
entries = Entry.objects.order_by('-date_posted') # date_posted descending order
```

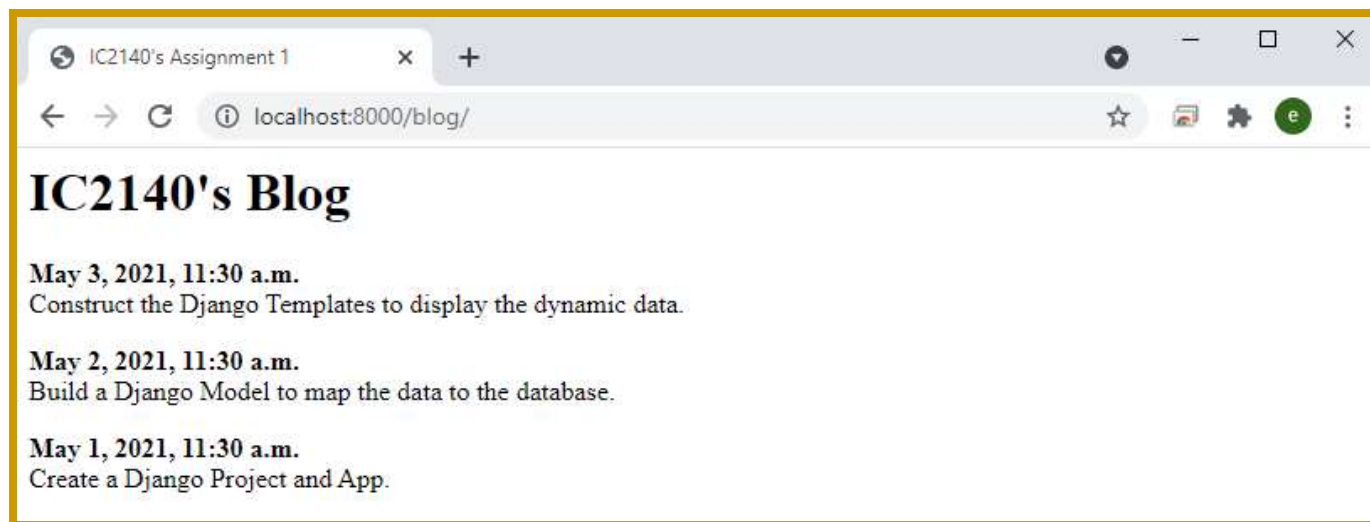
'-date_posted'

- date_posted is the field name in the Entry model
- The leading "-" indicates descending order. By default, fields without a leading "-" indicates ascending order

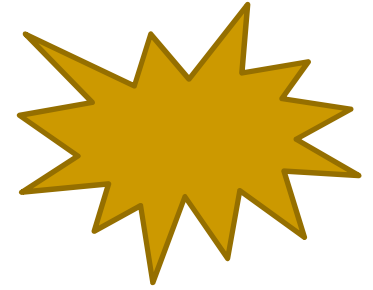
Display the entries in descending order

- 2) Open the below link in the Chrome Browser
(Assume that the server is running)

<http://localhost:8000/blog>



Worksheet - Task 3:



1) Suggest the method to retrieve the QuerySet of Entry model in the database

1. Retrieve the total number of Entry objects in the database
2. Retrieve Entry object(s) that the text field contains the "database" text
3. Retrieve Entry object(s) in 2020
4. Retrieve the first 2 Entry objects
5. Retrieve the Entry objects with primary key id=3

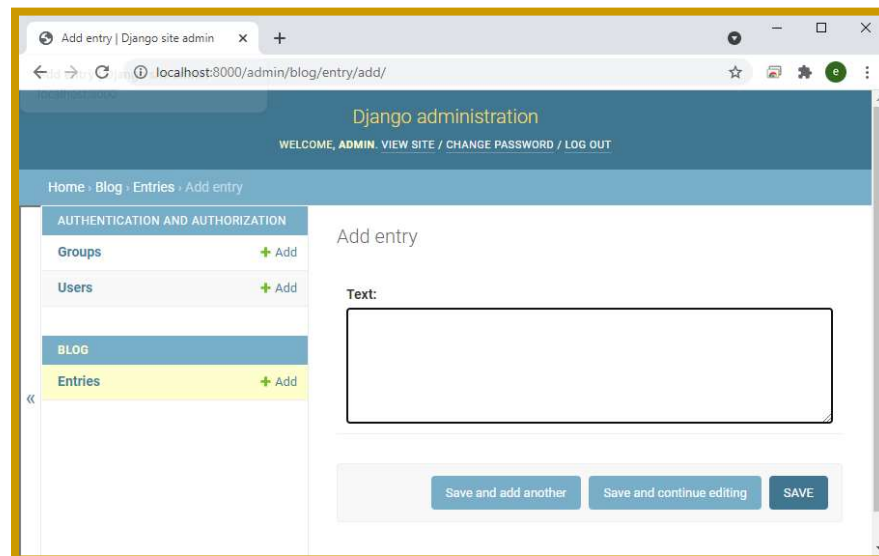
■ Making queries

- <https://docs.djangoproject.com/en/3.2/topics/db/queries/>

Django Form

HTML Form

- Collect information from users
- Is a group of one or more fields, including text boxes, checkboxes, radio button and so on
- Send data using **POST** requests
- Example: HTML form in Django Admin site for adding entry



The screenshot shows a web browser window displaying the Django Admin interface. The address bar shows the URL `localhost:8000/admin/blog/entry/add/`. The page title is "Add entry | Django site admin". The main header is "Django administration" with links for "WELCOME, ADMIN", "VIEW SITE", "CHANGE PASSWORD", and "LOG OUT". The breadcrumb trail is "Home > Blog > Entries > Add entry". The left sidebar contains a menu with "AUTHENTICATION AND AUTHORIZATION" (Groups, Users) and "BLOG" (Entries). The main content area is titled "Add entry" and features a large text input field labeled "Text:". At the bottom, there are three buttons: "Save and add another", "Save and continue editing", and "SAVE".

HTML Form

- Below is the simple HTML form, including 1 label and 1 text field for entering the username

Enter username:

- Below is the associated HTML code

```
<form action="/post_action_url/" method="post">
  <label for="username">Enter username: </label>
  <input type="text" name="username" id="username" value="Default username">
  <input type="submit" value="OK">
</form>
```

<form>...</form>

- Define in HTML as a collection of elements. E.g. <label> tag and <input> tag

action="" in <form> element

- The URL where data is to be send for processing when the submit button is pressed

method="" in <form> element

- The HTTP method used to send the data: POST or GET

type="" in <input> element

- Define what sort of widget will be display. E.g. text (textfield) and submit (button)

value="" in <input> element

- Define the initial value for the field

Django's Form Handling

- 1) Display the default form at the first time when it is requested by the user
- 2) Receive the data after user fill in the data and click the submit request
- 3) Clean and validate the data
 - ❑ Remove the invalid character
 - ❑ Convert them into consistent Python types
- 4) Invalid data => redisplay the form
- 5) Valid data => perform required actions (E.g. save the data, send an email, return the result of a search and etc.
- 6) Redirect the user to another/same page after all the actions are complete

Django ModelForm

- Very similar to declare a Model and share the same field and types
- Form data is stored in an application's `forms.py` file
- Save a great deal of time, effort, and code, because it will build a form, along with the appropriate fields and their attributes, from a Model class.

Create a ModelForm for adding entry

1) Declare a ModelForm

- a) Create a file `forms.py` under `mysite/blog` folder
- b) Copy the following code into it

```
from django.forms import ModelForm
from .models import Entry

class EntryForm(ModelForm):
    class Meta:                                # Attach additional information
        model = Entry
        fields = ('text', )
```

class Meta

- Include the associated model (Entry) and a list of model fields (text)

Render the form page when requested by user

1) Render the form associated with EntryForm in the `views.py`

- a) Open the `mysite/blog/views.py` file in the Visual Studio Code
- b) Append the following code to the bottom of import statement in the `views.py`

```
from .forms import EntryForm
```

- c) Append the `add()` function at the bottom of `views.py`

```
def add(request):  
    form = EntryForm()  
    context = {'form' : form}    # Pass the form to template  
    return render(request, 'blog/add.html', context)
```

Create a form page (add.html)

- 1) Create the template add.html
 - a) Create a file `add.html` under `mysite/blog/templates/blog` folder
 - b) Copy the following code into it

```
<HTML>
  <head>
    <title>IC2140's Assignment 1</title>
  </head>
  <body>
    <h1>IC2140's Blog</h1>
    <h2>Add new Entry</h2>
    <form method="POST" action="">
      {{ form.text }}
      <button>Submit</button>
    </form>
  </body>
</HTML>
```

Map URL to form page

- 1) Open the file "urls.py" under the [mysite/blog/](#) folder in Visual Studio Code
 - a) Append the "urlpatterns" variable with the following code (**Brown arrow**)

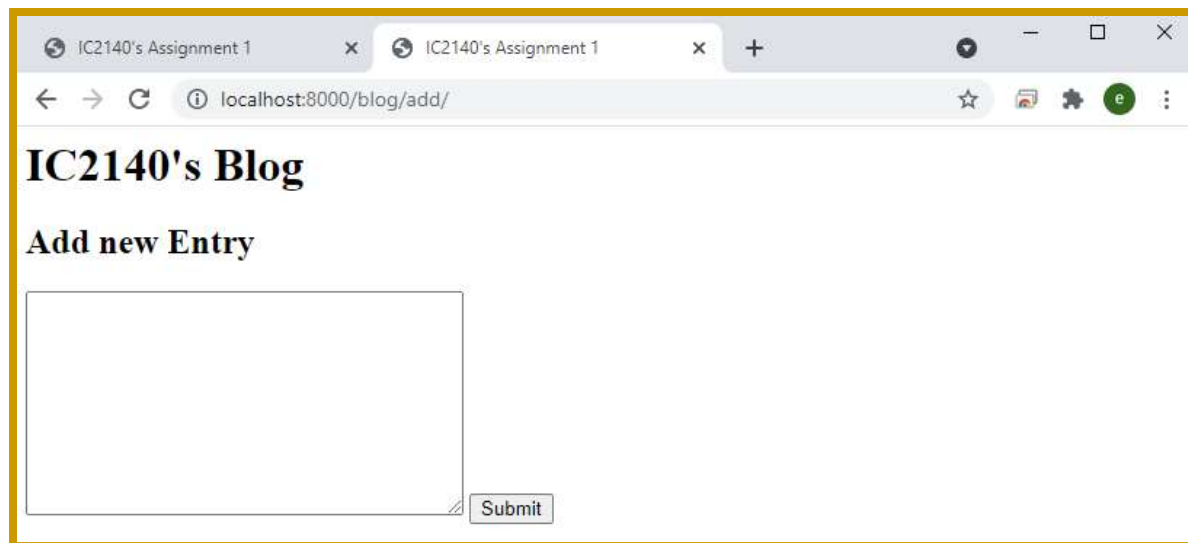


```
urlpatterns = [  
    path('', views.index),  
    path('blog/', views.blog),  
    path('blog/add/', views.add),  
]
```

Test the form page

- 1) Open the below link in the Chrome Browser
(Assume that the server is running)

<http://localhost:8000/blog/add>



The screenshot shows a web browser window with two tabs, both titled 'IC2140's Assignment 1'. The address bar displays 'localhost:8000/blog/add/'. The page content includes the heading 'IC2140's Blog' and the subheading 'Add new Entry'. Below the subheading is a large, empty rectangular text input field. At the bottom right of this field is a 'Submit' button.

Process the POST request

- Form data sent back to a Django website is processed by a `views.py`
- Tasks:
 - Check the POST request
 - If Yes, submit the data to the form
 - Otherwise, display the form
 - Check the form whether it is valid
 - If valid => Save it in the database and redirect to `index.html`
 - Otherwise, display the error

Process the POST request

- 1) Include the redirect function in the views.py
 - a) Open the [mysite/blog/views.py](#) file in the Visual Studio Code
 - b) Update the import statement with the following code (**Brown arrow**)



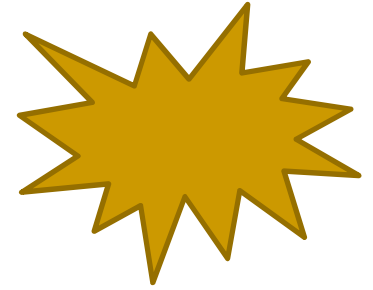
```
from django.shortcuts import render, redirect
from .models import Entry
from .forms import EntryForm
```

Process the POST request

- 2) Implement the processing POST request
 - a) Replace the add() method with the following code block

```
def add(request):  
    if request.method=='POST':  
        form = EntryForm(request.POST)          # POST request  
                                                # request.Post contains the data  
  
        if form.is_valid():  
            form.save()                          # Save the data in the database  
            return redirect('/blog/')            # Blog main page  
  
    else:  
        form = EntryForm()                      # Not POST request  
                                                # Create empty EntryForm  
  
    context = {'form' : form}                   # Pass the form to template  
    return render(request, 'blog/add.html', context)
```

Worksheet - Task 4:

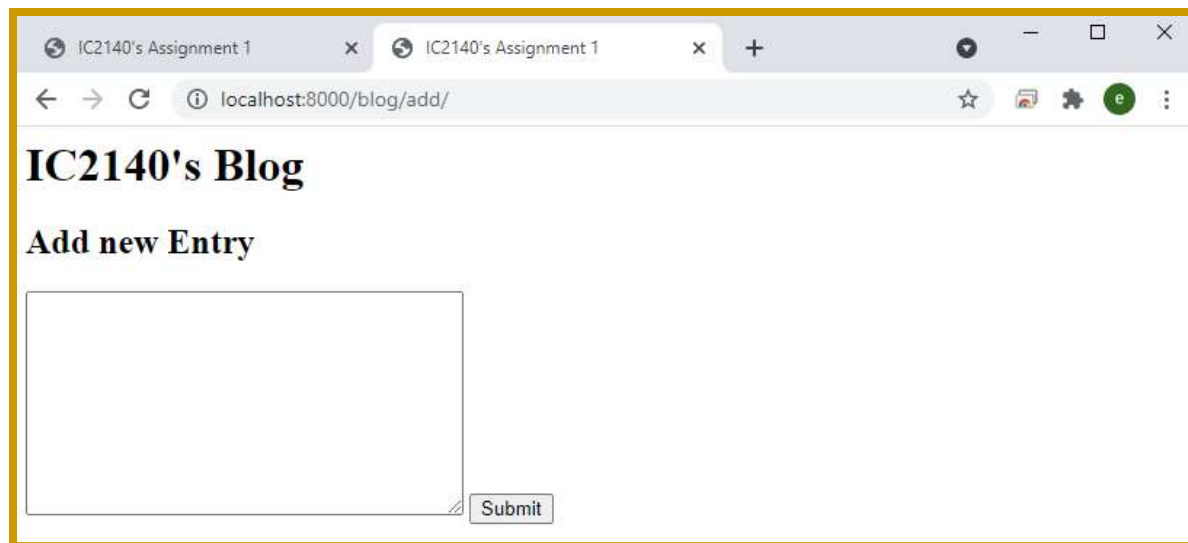


- 1) What is the difference between `render()` function and `redirect()` function in Django? Give an example to explain it.

Add the new entry

- 1) Open the below link in the Chrome Browser
(Assume that the server is running)

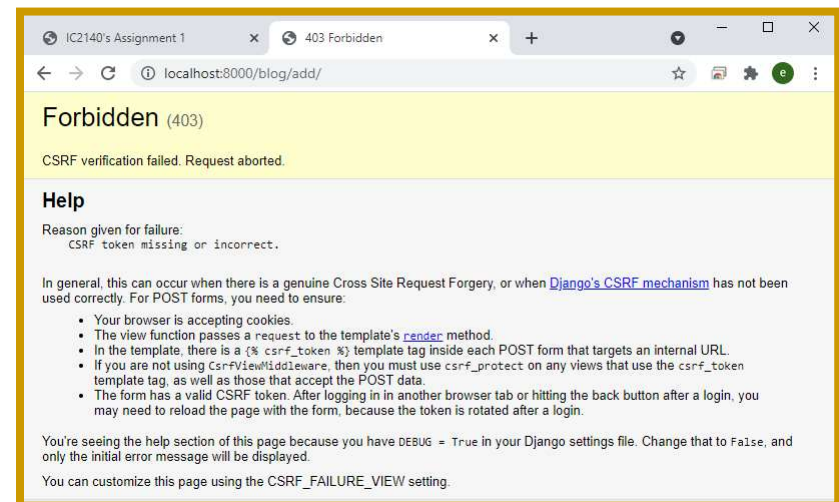
<http://localhost:8000/blog/add>



The screenshot shows a web browser window with two tabs, both titled 'IC2140's Assignment 1'. The address bar displays 'localhost:8000/blog/add/'. The page content includes the heading 'IC2140's Blog' and a sub-heading 'Add new Entry'. Below the sub-heading is a large, empty rectangular text input field. At the bottom right of this field is a 'Submit' button.

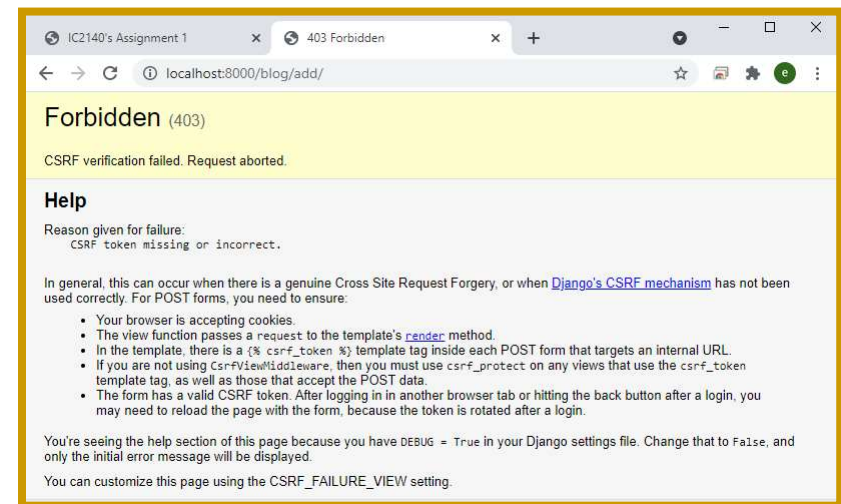
Add the new entry

- 1) Input the text "Build a HTML form for processing and responding the input." in the Text area
- 2) Click "Submit" button
 - a) Get the error "Forbidden....CSRF verification failed. Request aborted."



Cross Site Request Forgeries (CSRF)

- The error is generated by the Cross Site Request Forgeries (CSRF)
- Prevent the attacker from inducing users to perform actions that they do not intend to perform
- Default: Activated



Use the CSRF token in the template

- 1) Include the CSRF token inside the <form> element in any template using POST request
 - a) Open the [mysite/blog/templates/add.html](#) file in the Visual Studio Code
 - b) Add the following code (**Brown arrow**) after the line <form method="POST" action="">



```
<form method="POST" action="">
{% csrf_token %}
{{ form.text }}
<button>Submit</button>
```

- c) Refresh the webpage

Add the new entry again

1) Input the text "Build a HTML form for processing and responding the input." in the Text area

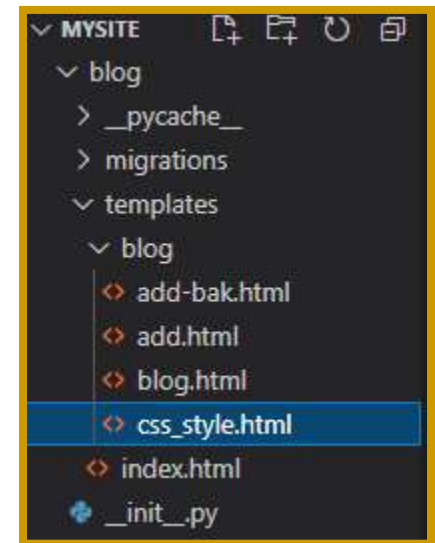


2) Click "Submit" button



Worksheet - Task 5:

- 1) Assume that there has a template file "css_style.html" under the [mysite/blog/templates/blog](#) folder. Show the steps how to implement in order to render it when user requests it using the below link http://localhost:8000/blog/css_style/



Cascading Style Sheets (CSS)

- Describe the look and formatting of a website written in HTML
- Bootstrap is one of the most popular HTML and CSS frameworks for developing beautiful websites
 - <https://getbootstrap.com/>

Install Bootstrap in template

- 1) Open the [mysite/blog/templates/blog/blog.html](#) file
- 2) Enable HTML5
 - 1) Add the following code (**Brown arrow**) before `<HTML>` tag



```
<!doctype html>  
<HTML>
```

- 3) Add the following code inside `<head>` section before the `<title>...</title>` tag

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-wEmeIV1mKuiNpC+IOBjI7aAzPcEZeedi5yW5f2yOq55WWLwNGmvvx4Um1vskeMj0" crossorigin="anonymous">
```


Apply Bootstrap CSS in template

- 1) Replace the `<boby>...</boby>` code block with the following code

```
<body>
  <h1 class="display-3 p-3 mb-2 bg-danger text-white">IC2140's Blog</h1>
  {% for entry in entries %}
    <article>
      <span class="d-block p-2 bg-primary text-white">{{ entry.date_posted }}</span>
      <div class="p-3 mb-2 bg-secondary text-white">{{ entry.text }}</div>
    </article>
  {% endfor %}
</body>
```

`<span class="d-block p-2 bg-primary text-white">`

- The `<span>` element will be styled according to `class="d-block"`, `class="p-2"`, `class="bg-primary"` and `class="text-white"`

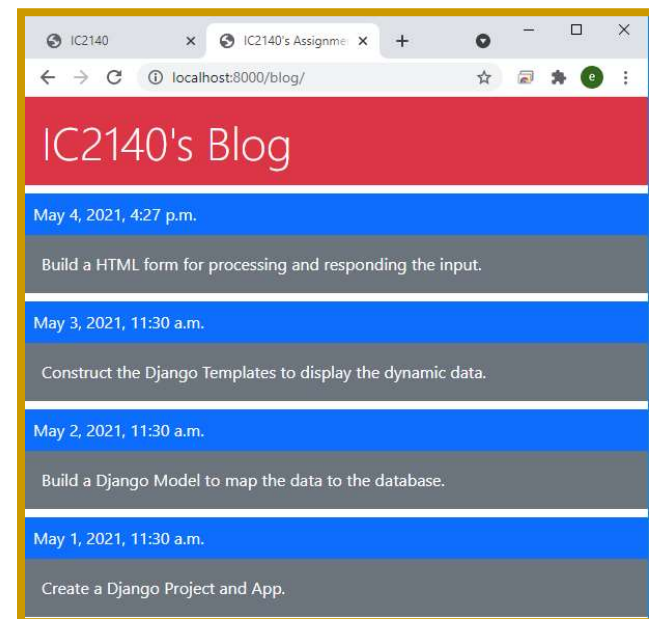
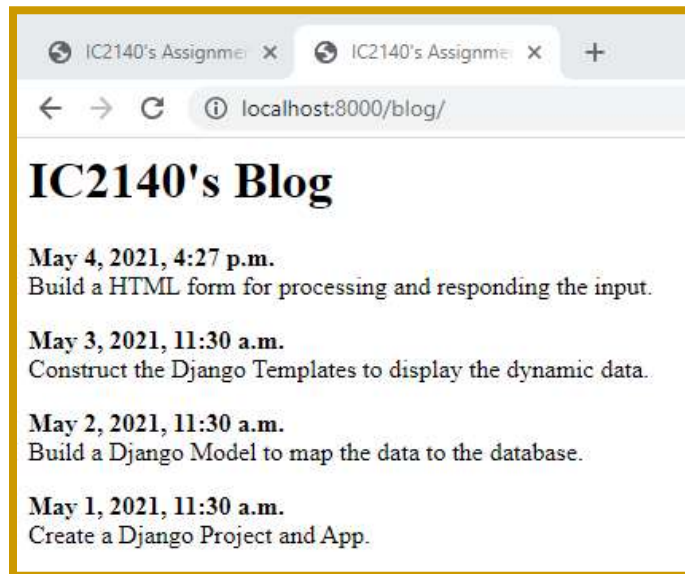
Bootstrap CSS

d-block	IC2140
p-2	IC2140
bg-primary text-white	IC2140
d-block p-2 bg-primary text-white	IC2140

Apply Bootstrap CSS in template

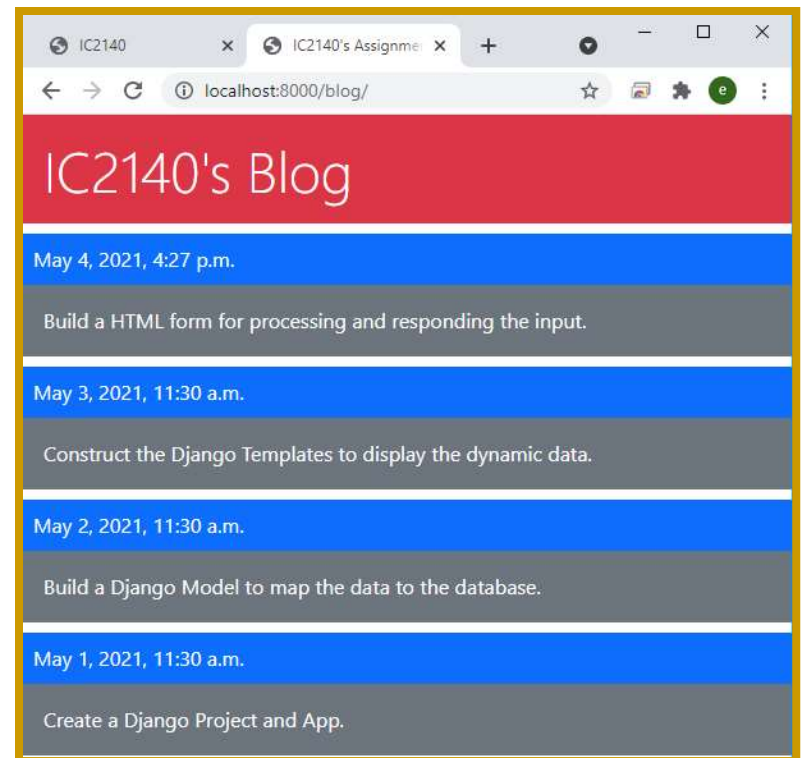
- 2) Open the below link in the Chrome Browser
(Assume that the server is running)

<http://localhost:8000/blog>



Example: Create Blog website

- Complete the Blog website
- Show the demonstration



Django Assignment 1: Online Inventory Management System

- Develop online inventory management system (app)
 - ❑ List all the inventory items including the item no., type, name, description, brand, unit price, stock and created date
 - ❑ Allow the user to update the inventory items in the Admin page
 - ❑ Show the inventory item(s) that the unit price is more than \$4,000
 - ❑ Show the inventory item(s) that the created date is more than 5 years
 - ❑ Show the summary of total no. of inventory in each type and each band in table style
 - ❑ Apply the Bootstrap CSS to make a beautiful interface
 - ❑ Import the sample data "item_sample.xls" to the app.

Reference: <https://docs.djangoproject.com/en/dev/topics/db/aggregation/>

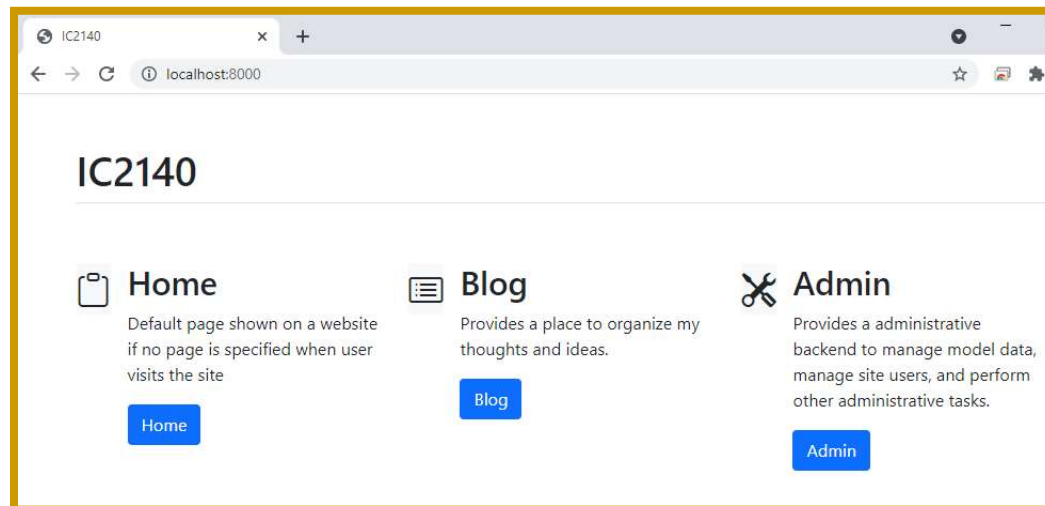
Django Assignment 1: Online Inventory Management System

■ Procedures

- 1) Create "inventory" app
- 2) Add "inventory" app to "INSTALLED_APPS" variable
- 3) Create the model in model.py
- 4) Register the model in admin.py
- 5) Add 5 sample items in the "admin" page
- 6) Create index page in template folder to list the inventory
- 7) Render the index page
- 8) Map URL to index page in inventory urls.py
- 9) Update the main entry point in the main urls.py

Django Assignment 2:

- Create navigation menu to index.html
 - ❑ Add a link pointing to each web page (horizontal or vertical bar)
 - Home page
 - Blog page (blog/)
 - Django Admin page (admin/)

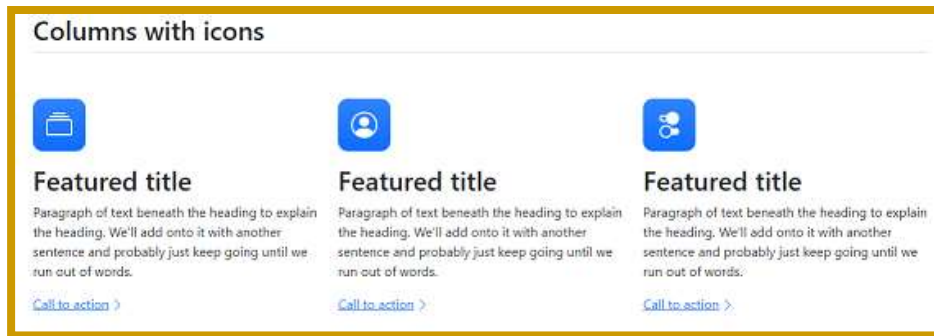


Cont.

Django Assignment 2:

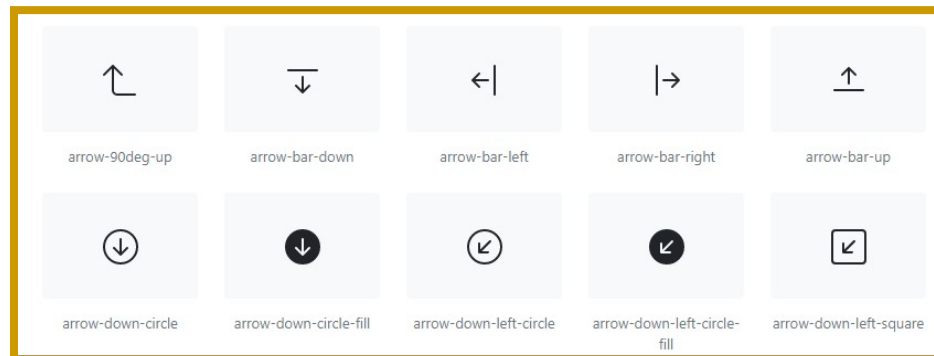
- Layout examples

- <https://getbootstrap.com/docs/5.0/examples/features/>



- Bootstrap Icons

- <https://icons.getbootstrap.com/>



Cont.

Django Assignment 2:

- Install Bootstrap Icons in the template.
 - Add the following code inside <head> section before the <title>...</title> tag

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.5.0/font/bootstrap-icons.css">
```

- Embedded the alarm icon with using "bi-" prefix

```
<i class="bi-alarm" style="font-size: 2rem;"></i>  
<i class="bi-tools" style="font-size: 2rem;"></i>
```



alarm

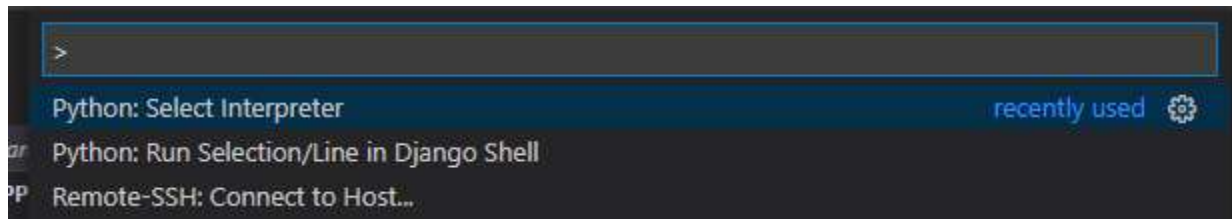


tools

Appendix

Enable the Django Python Interpreter

- 1) Press "CTRL"+"Shift"+"p" and select "Python: Select Interpreter"



- 2) Click "Enter interpreter path..."
- 3) Click "Find..."
- 4) Select the Python interpreter in the "C:\django-env\Scripts\python.exe"